158.337 Database Development

Week 7 – Procedural Language SQL

Read Chapter 7 and 8 again

Chapter 7 feeds into Chapter 8; hence for proper understanding of PL/SQL concepts, you should have clarity on SQL language's dos and don'ts. As a first step, you should be comfortable with SQL concepts in Chapter 7 before you proceed into more advanced concepts of PL/SQL in Chapter 8.

Unfortunately, SQL does not support conditional execution of statements (IF…THEN….ELSE) or repetitive execution of statements (LOOP statements) which are typically encountered in a programming environment. So, we need some procedural type of programming language to work with more flexibility in the database. Procedural Language SQL (PL/SQL) makes it possible to use and store procedural code by merging SQL statements with traditional programming constructs.

This week we start with PL/SQL. PL/SQL is like any other programming language – it too has built in syntax and rules that will determine how the programming statements will work together. It is strongly typed language, i.e. each variable needs to be explicitly declared with its specific data type before it can be used.

PL/SQL resides in two environments – Client and Server. So, first and foremost we need to ensure that server output is visible. This is done by writing the statement - SET SERVEROUTPUT ON - at the start of your session. This statement will ensure that your client console receives messages from the server side (Oracle DBMS). All named PL/SQL blocks are stored in the database server and are meant to be reused. Therefore the named blocks (e.g., procedures, functions, triggers) are used by application programs during execution time. If a procedure is named (and hence it is not anonymous), then it can be called by its name from other program units. However if it is not named (and hence is anonymous), then no program unit can call it. Therefore, we name our PL/SQL blocks so that they can be saved on the Oracle server and can be continually referenced, just as we would reference other database objects, such as tables and sequences.

You must supplement Chapter 8 reading with lecture slide examples. I have used many small examples (both from the textbook and also others) to demonstrate the functionality of other aspects of embedded SQL queries in Pl/SQL.

The lecture slides cover simple examples to demonstrate PL/SQL functionality. The lecture slides are self-explanatory with examples on usage of variety of constructs. These illustrations are exemplars of commonly used constructs for performing various tasks. I urge you all to practice writing the scripts provided on slides to gain more understanding of implementation aspects.

This week we will concentrate on writing procedures, functions, cursors and triggers. Procedures and functions are quite similar in concepts –and are both used extensively in program units. So here we will work with IF…THEN ….. ELSEIF …. END IF statements, CASE statements, iterative statements (LOOPS – for, while), etc…

We will also look at exception handling scenarios. It is important to safeguard your program from unforeseen errors. Here we will use some built-in exceptions and also user-defined exceptions. RAISE_APPLICATION_ERROR is a special built-in procedure provided by Oracle. It allows us to create meaningful error messages which suit our problem domain.

Cursors and triggers are exciting things, and are advanced concepts in PL/SQL. We will discuss them next week.